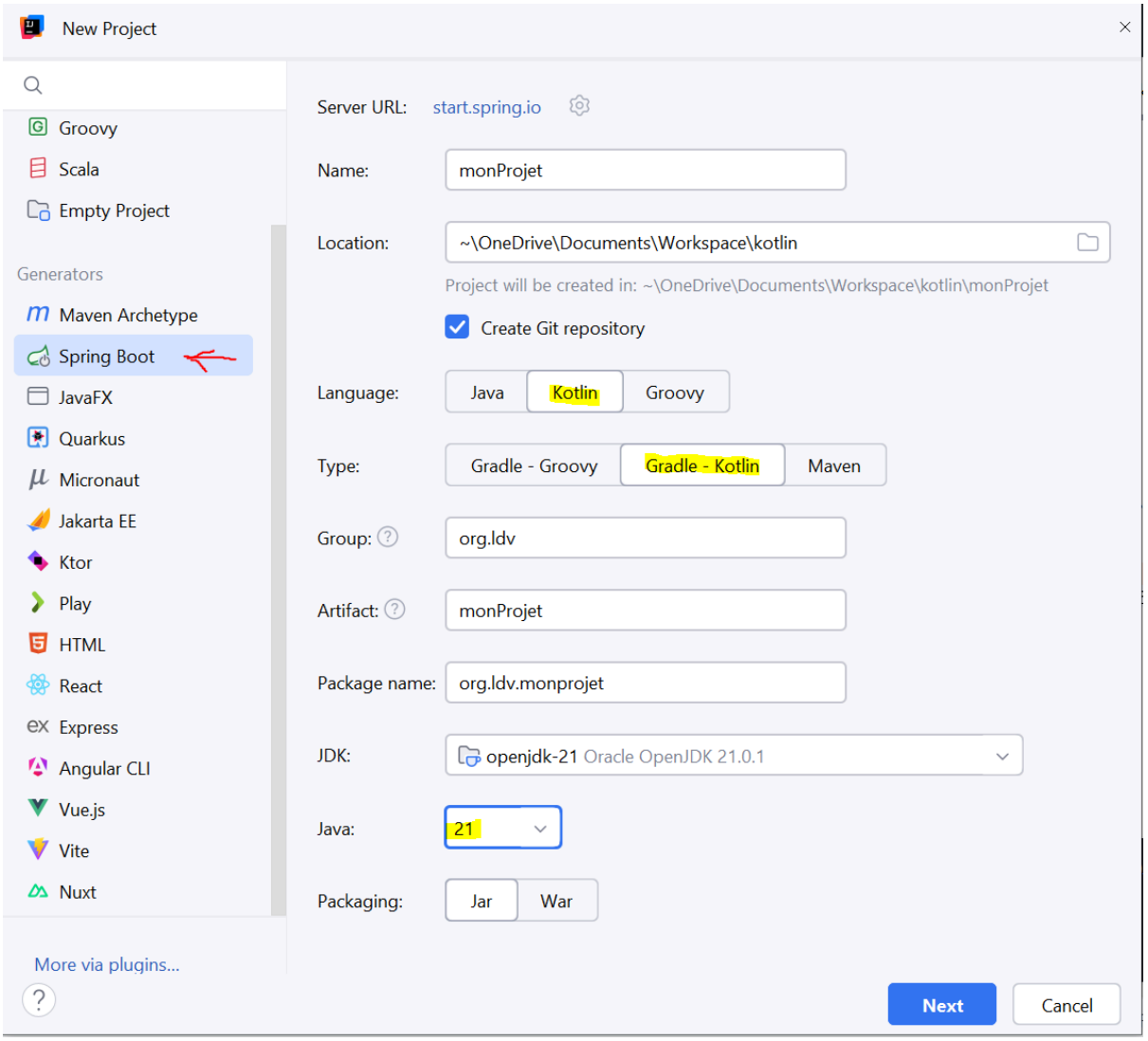
Projet 1 : Sprint 1 Création du noyau

Création du projet

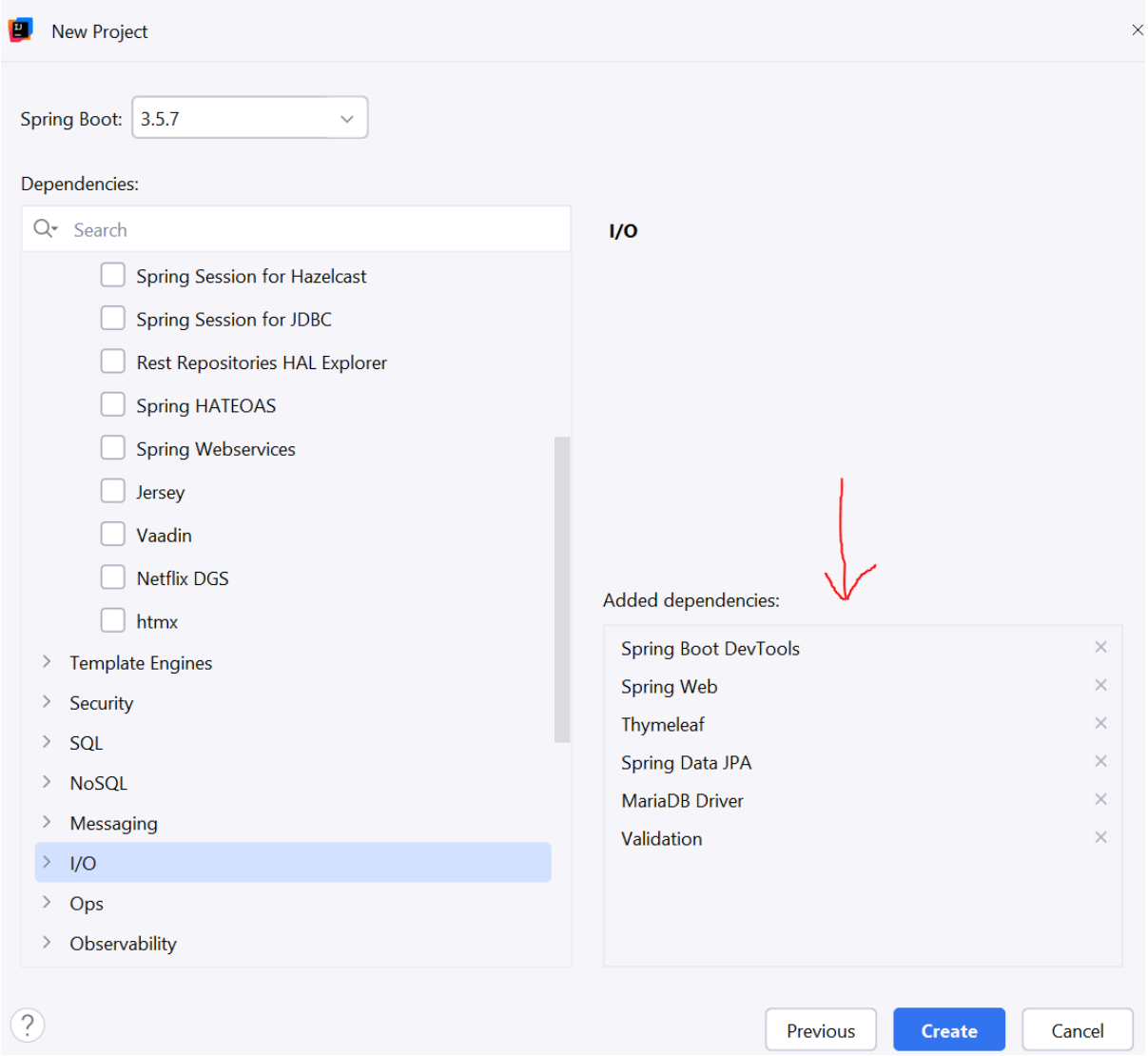
Dans **IntelliJ IDEA**, créez un nouveau projet :

File → New → Project



Remplacez `monProjet` par le nom de votre projet (par exemple : `webmonsters` ou `ekommerce`).

Sélection des dépendances



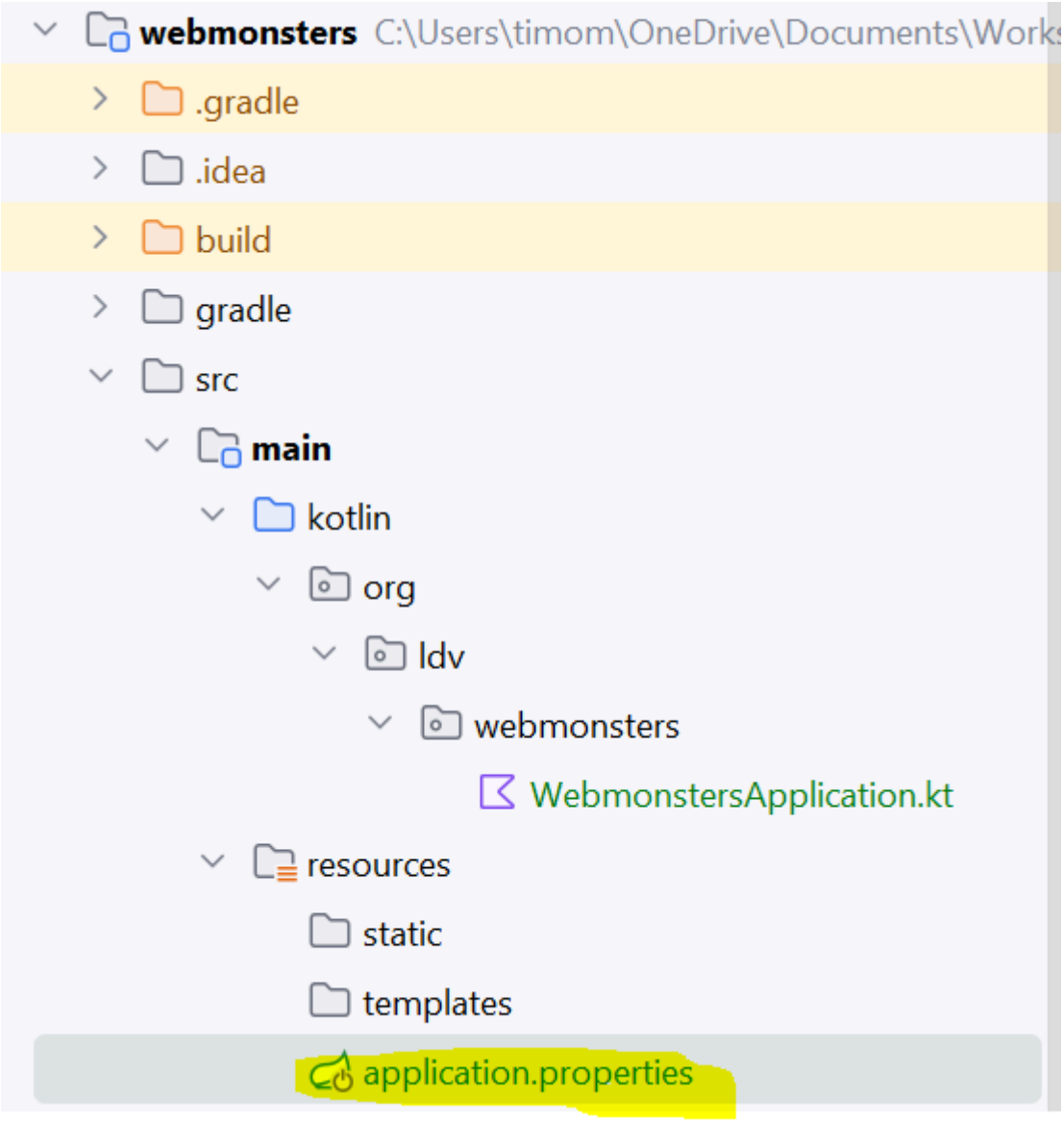
Lors de la création du projet, sélectionnez les dépendances suivantes :

- **Spring Web** : permet de créer des contrôleurs et de gérer les routes HTTP.
- **Thymeleaf** : moteur de templates pour générer les pages HTML côté serveur.
- **Spring Data JPA** : facilite l'accès à la base de données à l'aide d'entités et de repositories.
- **MariaDB Driver** : permet la connexion à une base de données MariaDB/MySQL.
- *(Optionnel)* **Spring DevTools**: facilite le redémarrage automatique du serveur lors des modifications (pratique pour le développement).
- *(Optionnel)* **Validation** : Permet de faire plus facilement la validation des formulaires.

💡 *Ces dépendances sont nécessaires pour construire un projet web complet avec une base de données et un système de templates.*

Configuration de la connexion au SGBD

Ouvrez le fichier `src/main/resources/application.properties` :



application.properties est un fichier qui sert à configurer notre application notamment la connexion au serveur de la BDD.

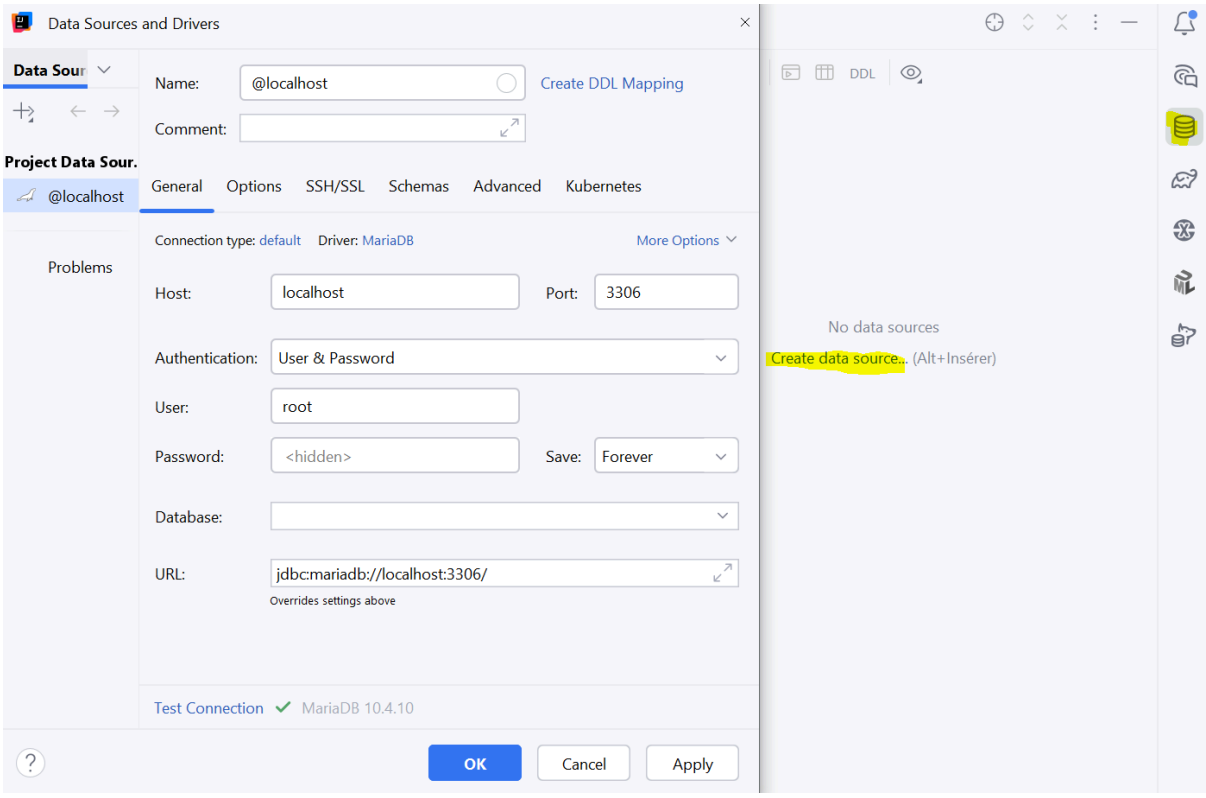
```
spring.datasource.url=jdbc:mariadb://localhost:3306/db_webmonsters?c
reateDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.jpa.hibernate.ddl-auto=create-drop
```

Explications :

- `spring.datasource.url` : adresse de connexion à la base de données
- `spring.datasource.username` / `password` : identifiants d'accès
- `spring.jpa.hibernate.ddl-auto` :
  - `create-drop` → crée la base au démarrage et la supprime à l'arrêt (utile pour les tests pendant le développement du projet)
  - pour un usage réel, utilisez `update` ou `validate`.

👉 Modifiez cette configuration pour vous connecter à votre **SGBD distant**.
Démarrez ensuite le projet pour vérifier la connexion : si le serveur démarre sans erreur, la configuration est correcte.

Je vous recommande aussi de configurer le datasource pour plus tard.



Mettre votre projet sur Git

Avant d'ajouter le projet sur git je vous invite à créer un fichier readme.md a la racine du projet inteliji.

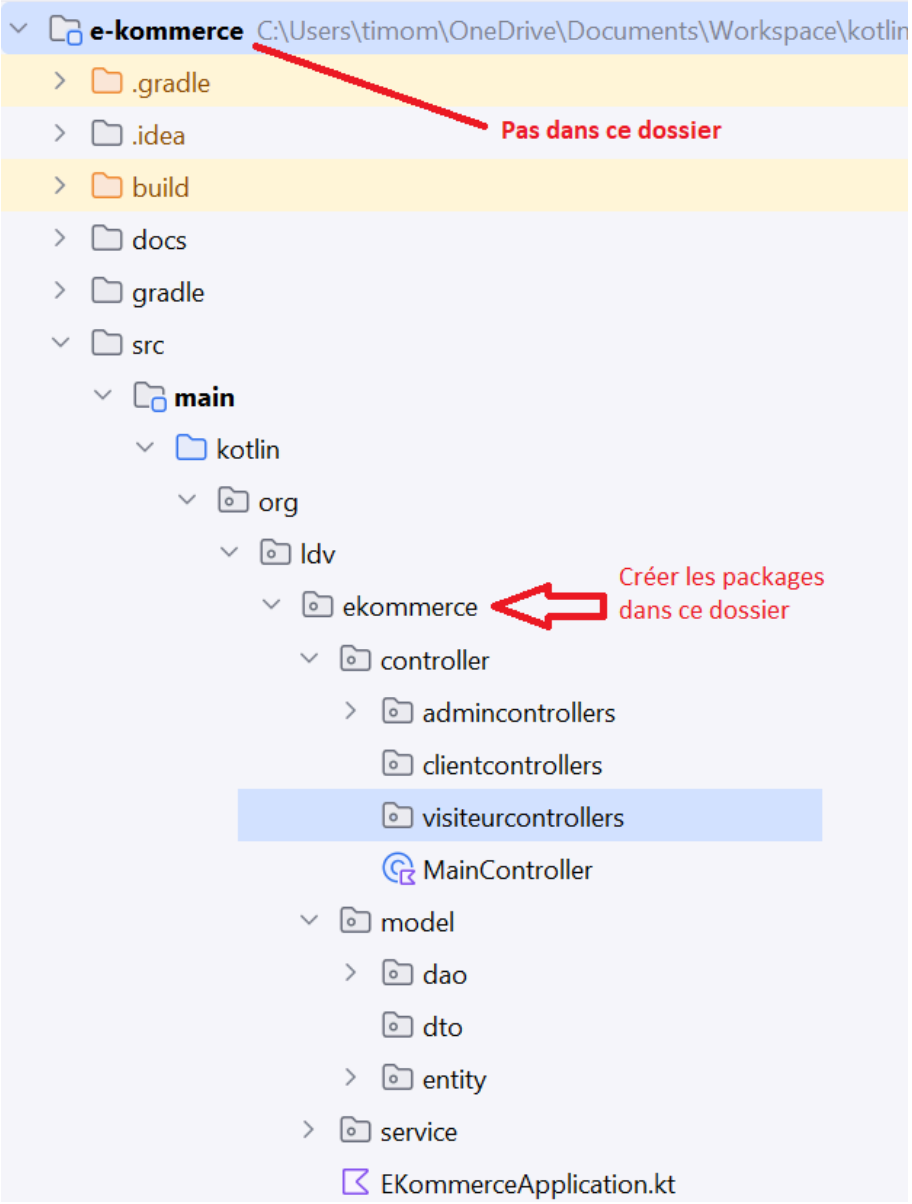
Une fois votre projet mis sur Git Hub mettre à jour le tableau :

https://docs.google.com/spreadsheets/d/1k5obWQC_mQaaau6rc8bFcNpB8gBSuV4b/edit?usp=sharing&ouid=109629770337688255435&rtpof=true&sd=true

Organisation de l'arborescence

Je vous propose l'organisation suivante au niveau des dossiers du projet :

Organisation de la partie Kotlin



Explications :

- controller :
  Contient les **contrôleurs**, c'est-à-dire les classes qui **gèrent les requêtes HTTP** (via les annotations `@GetMapping`, `@PostMapping`, etc.).
  Ils reçoivent les requêtes des utilisateurs, appellent les services ou dao

nécessaires et renvoient la réponse ou la vue correspondante (ex. une page HTML via Thymeleaf).

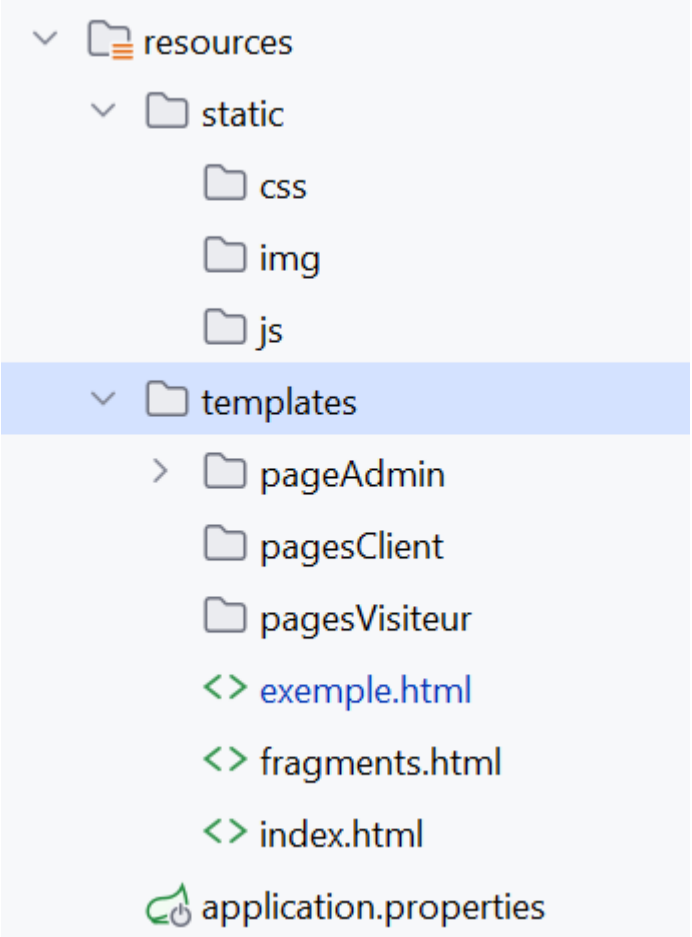
- service :
  Contient la **logique métier** du projet.
  Ces classes servent à **éviter la répétition de code** et à **centraliser les traitements spécifiques**.
  Par exemple :
  - Remplir automatiquement les tables de la base de données ;
  - Convertir des données en différents formats (JSON, CSV, PDF, etc.) et inversement ;
  - Appliquer des règles de gestion ou des calculs avant l'enregistrement en base (ex : gestion du panier).

  👉 En résumé : un service réalise les **tâches concrètes** nécessaires au fonctionnement de l'application.
- dao (ou `repository`, selon les projets) :
  Contient les **classes d'accès aux données**.
  C'est ici que l'on définit les **interfaces** permettant de communiquer avec la base de données (souvent via `JpaRepository` ou `CrudRepository`).
  Ces classes sont responsables des **opérations CRUD** : *Create, Read, Update, Delete*.

- entity :
  Contient les **classes métiers représentant les tables de la base de données**.
  Chaque entité correspond à une table, et chaque attribut de la classe correspond à une colonne.
  Ces classes sont annotées avec `@Entity`, `@Id`, `@Column`, etc.

- dto *(Data Transfer Object)* :
  Facultatif mais souvent utile.
  Un DTO est une **classe utilisée pour transférer des données** entre différentes couches de l'application sans exposer directement les entités.
  Il est particulièrement pratique lorsque :
  - Un **formulaire ne correspond pas exactement** à la structure de la classe métier/entité ;
  - Les **règles de validation** diffèrent par exemple entre la création et la modification d'un objet ;
  - On souhaite **limiter les informations envoyées au client** pour des raisons de sécurité ou de performance.

Organisation des ressources et templates :



Suggestions :

- Vous pouvez créer des sous-dossiers dans le dossier *img* (produits, categories, marques, monstres ...)
- Si vous avez besoin de faire des headers et/ou footer très différents en fonction des pages ou des utilisateurs, je vous invite à créer un dossier *fragments* dans le dossier templates.

Explication :

- Le dossier *pagesAdmin* a pour but de recevoir toutes les pages qui concernent l'admin.
- Le fichier *fragments.html* contient le header et footer que l'on souhaite réutiliser sur toutes nos pages.

  Exemple :

```
<div th:fragment="footer" class="container">
<footer class="">
    <!--Votre pied de page-->
</footer>

</div>
```

- Le fichier *base.html* est un exemple de page qui utilise le système de templating de thymeleaf. Quand je dois créer une nouvelle page je fais une copie de ce fichier.

Exemple :

```
<!doctype html>

<html lang="fr" xmlns:th="http://www.thymeleaf.org">

<head>

   <meta charset="utf-8">

   <meta name="viewport" content="width=device-width,
initial-scale=1">

   <title>E-komerce</title>

   <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bo
otstrap.min.css" rel="stylesheet"


integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIA
vNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">

</head>

<body>
```

```
<div class="min-vh-100">
   <header th:replace="fragments :: header"></header>
   <main>
       <div class="container" th:replace="::content">


       </div>
   </main>
</div>
<div th:replace="fragments :: footer"></div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/boot
strap.bundle.min.js"

integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcX
K/BmnVDxM+D2scQbITxI"
       crossorigin="anonymous"></script>
</body>
</html>
```

Les th:replace permettent de remplacer mon header et footer par mes fragments.

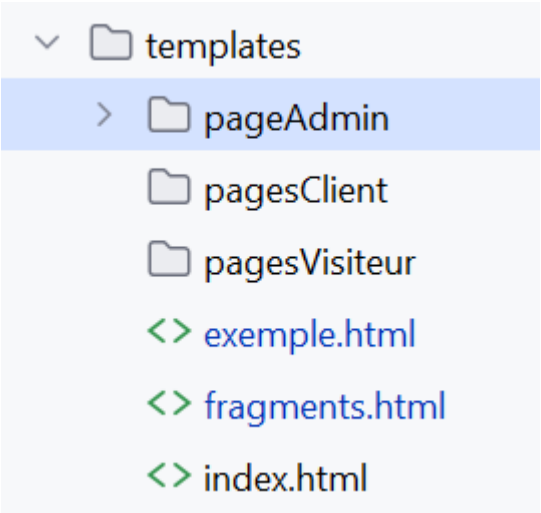
Créer vos fragments (footer, header) .

Créer un fichier base.*html* :

- Importer bootstrap
- Importer votre feuille de style CSS(que vous devais mettre dans static/css)
- Utiliser vos fragments pour le header et footer et un emplacement pour le content

Pour tester si votre design correspond bien à ce que vous souhaitez on va créer la page home :

- **Créer une page index.html**



- **Pour l'instant dans le fichier index.html :**

```
<!doctype html>
<html th:replace="base :: html" lang="fr"
xmlns:th="http://www.thymeleaf.org">
<div th:fragment="content">
   <main class="py-5">
       <div class="container">
           <h1 class="mb-5 text-center">Bienvenue sur
<strong>E-Kommerce</strong></h1>


       </div>
   </main>
</div>

</html>
```

- **Dans le dossier controller créer une classe MainController**
- **Voici le code du *MainController***

```
@Controller
class MainController (){

/**
 * Méthode permettant d'afficher la page d'accueil de l'application.
 * @return le chemin vers le template a partir du dossier
ressources/templates (on ne marque pas le .html)
 */
@GetMapping("/e-kommerce")
fun home():String{
        return "index"
    }

}
```

Je vous invite à changer le GetMapping pour que le chemin soit approprié pour votre projet.

Ensuite redémarrer le projet et faite un test dans le navigateur :

http://localhost:8080/leCheminDuGetMapping

Faire les ajustements nécessaires sur le fichier *index.html* et vos fragments.

Quand vous êtes satisfait de votre template mettez à jour votre fichier ***exemple.html***.

Modifier la page index.html pour qu' elle corresponde vraiment à votre page d'accueil, si vous avez besoin d'afficher des produits ou autre sur la page je vous invite pour l'instant à le faire en statique (c'est à dire le faire directement en HTML) vous pouvez répéter le même produit plusieurs fois.

Création des autres pages simples

Créez les pages suivantes :

- `a-propos.html`
- `contact.html`
- `inscription.html`
- `produits.html`

- `rgpd.html`

Pour chacune :

1. Créez le fichier dans `templates/pagesVisiteur`
2. Ajoutez une méthode dans `MainController` (avec un GetMapping différent)